

A
Project Report On
JOB PORTAL DATABASE

Submitted for partial fulfilment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY IN
COMPUTER SCIENCE AND ENGINEERING (AI&ML)

by
E SAI PRIYA-23K81A6620

Under the guidance of
Mrs.K. SWETHA
ASSISTANT PROFESSOR



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING(AI&ML)

St. MARTIN'S ENGINEERING COLLEGE.

UGC Autonomous

Affiliated to JNTUH, Approved by AICTE

Accredited by NBA & NAAC A+, ISO 9001-2008 Certified

Dhulapally, Secunderabad-500 100

www.smec.ac.in



St. MARTIN'S ENGINEERING COLLEGE

An autonomous Institute

NBA & NAC A+ Accredited

Dhulapally, Secunderbad-500100



CERTIFICATE

This is to certify the project entitled “ **JOB PORTAL DATABASE** ” is being submitted by **E SAI PRIYA,23K81A6620** in fulfilment of the requirement for the award degree of Bachelor of Technology in ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING is recorded of bonafide work carried out by them. The result embodied in this report have been verified and found satisfactory.

Project Internal Examiner

Mrs.K.SWETHA

Assistant professor

Department of CSE(AIML)

Signature of HOD

Dr.M.GOWTHAM

Professor and Head of Department

Department of CSE(AIML)



St. MARTIN'S ENGINEERING COLLEGE

An autonomous Institute

NBA & NAC A+ Accredited

Dhulapally, Secunderabad-500100



DECLARATION

I, the student of “**Bachelor of Technology**” in “**Computer Science and Engineering**”, session: 2023 – 2027, St. Martin’s Engineering College, Dhulapally, Kompally, Secunderabad, hereby declare that the work presented in the Project Work entitled as “**JOB PORTAL DATABASE**” is the outcome of my own bonafide work and is correct to the best of my knowledge and this work has been undertaken taking care of Engineering Ethics. This result embodied in this project report has not been submitted in any university for award of any degree.

E SAI PRIYA - 23K81A6620

UGC AUTONOMOUS

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose encouragement and guidance have crowded our efforts with success.

First and foremost, we would like to express our deep sense of gratitude and indebtedness to our College Management for their kind support and permission to use the facilities available in the Institute.

We especially would like to express our deep sense of gratitude and indebtedness to **Dr. P. SANTOSH KUMAR PATRA**, Group Director, St. Martin's Engineering College Dhulapally, for permitting us to undertake this project.

We wish to record our profound gratitude to **Dr. M. SREENIVAS RAO**, Principal, St. Martin's Engineering College, for his motivation and encouragement.

We are also thankful to **Dr. M. GOWTHAM**, Head of the Department, CSE(AI&ML), St. Martin's Engineering College, Dhulapally, Secunderabad, for his support and guidance throughout our project.

We would like to express our sincere gratitude and indebtedness to our project supervisor **Mr. K.SWETHA** Assistant Professor, Department of Computer Science and Engineering(AI&ML), St. Martins Engineering College, Dhulapally, for his/her support and guidance throughout our project.

Finally, we express thanks to all those who have helped us successfully completing this project. Furthermore, we would like to thank our family and friends for their moral support and encouragement. We express thanks to all those who have helped us in successfully completing the project.

E SAI PRIYA - 23K81A6620

ABSTRACT

The Job Portal Database Management System is designed to facilitate seamless interactions between job seekers and employers. This system offers a comprehensive platform where job seekers can create profiles, upload resumes, and search for job listings tailored to their skills and interests. Employers, on the other hand, can post job vacancies, search for potential candidates, and manage applications efficiently. The database is structured to store detailed information on both job listings and candidates, ensuring quick retrieval and smooth navigation through the platform's functionalities. The core of this project is a robust database schema that integrates various entities such as users, job postings, applications, and companies. The system utilizes SQL for managing data storage, retrieval, and manipulation, ensuring data integrity and consistency. Advanced queries are implemented to facilitate complex searches and matching algorithms that pair job seekers with suitable job opportunities based on multiple criteria, including experience, education, and location preferences. This project emphasizes the importance of data normalization, indexing, and transaction management to maintain high performance and reliability. Additionally, the project incorporates security measures to protect sensitive user information, including encryption for password storage and secure login mechanisms. User roles are clearly defined to control access levels, ensuring that personal data and company information remain confidential. The Job Portal Database Management System also includes features for monitoring user activities and generating analytical reports to help employers and job seekers make informed decisions. This comprehensive approach aims to create a user-friendly, efficient, and secure platform for the job market.

UGC AUTONOMOUS

TABLE OF CONTENTS

TITLE	PAGE NO
Acknowledgement	iii
Abstract	iv
Table of Contents	v
List of Figures	vi
Introduction	1
System Analysis	2
Algorithm	3
Source Code	7
System Testing	11
Output Screen	12
System Requirements	14
Conclusion	17
Future Enhancement	18
References	19

UGC AUTONOMOUS

INTRODUCTION

A Database Management System (DBMS) mini-project on a Job Portal Database aims to develop a robust and efficient database to manage various aspects of a job portal. This project involves designing and implementing a database that can handle the storage, retrieval, and manipulation of job listings, job seeker profiles, employer information, and application records. The primary objective is to create a scalable and reliable database system that supports the functionalities of a typical job portal, including job search, profile management, job application, and employer postings.

The project begins with the requirement analysis phase, where the essential features and functionalities of the job portal are identified. This includes understanding the relationships between different entities such as job seekers, employers, job listings, and applications. The database schema is then designed using Entity-Relationship (ER) diagrams to model these relationships accurately. Key considerations during this phase include ensuring data integrity, avoiding redundancy, and optimizing performance. The schema typically includes tables for users (job seekers and employers), jobs, applications, and possibly auxiliary tables for skills, locations, and job categories.

Implementation involves creating the database using a DBMS like MySQL, PostgreSQL, or Oracle. SQL queries are used to create tables, insert data, and establish relationships between tables through primary and foreign keys. The project also includes developing user interfaces or APIs to interact with the database, enabling users to perform operations such as registering, logging in, posting jobs, searching for jobs, and applying for jobs. The final phase involves testing the database to ensure it meets the specified requirements and performs efficiently under various scenarios. This project not only enhances understanding of database design and management but also provides practical experience in building a real-world application.

SYSTEM ANALYSIS

EXISTING SYSTEM

1. **User Authentication:** Secure login system for job seekers and employers, ensuring data privacy and role-specific access.
2. **Job Listings Management:** Employers can post, update, and delete job listings; job seekers can search and apply for jobs.
3. **Profile Management:** Job seekers can create, update, and manage their profiles, including resumes and skills.
4. **Application Tracking:** Both employers and job seekers can track the status of job applications, with notifications for updates.
5. **Data Analytics and Reporting:** Generate reports on job market trends, user activity, and application statistics for administrative insights.

PROPOSED SYSTEM

1. **User Registration and Authentication:** A secure module for users to create accounts, log in, and manage profiles.
2. **Job Listings Management:** A feature for employers to post, update, and manage job openings, including job descriptions and requirements.
3. **Job Search and Application:** A search function for job seekers to find relevant job postings and apply directly through the portal.
4. **Resume and Profile Management:** Tools for job seekers to upload resumes, update personal information, and highlight skills and experiences.
5. **Admin Dashboard:** An interface for administrators to oversee the system, manage users, and ensure the integrity of job listings and applications.

ALGORITHM

1. Define Requirements

Identify the core functionalities:

User registration and login

Profile management

Job posting

Job search

Job application

Admin management

2. Design the Database Schema

Tables and relationships:

Users (user_id, name, email, password, role [employer/applicant])

Profiles (profile_id, user_id, resume, skills, experience)

Jobs (job_id, employer_id, job_title, job_description, location, salary, posted_date)

Applications (application_id, job_id, applicant_id, application_date, status)

Admins (admin_id, name, email, password)

3. Implement the Database Schema

Using SQL:

```

CREATE TABLE Users (
    user_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    email VARCHAR(100) UNIQUE,
    password VARCHAR(100),
    role ENUM('employer', 'applicant')
);

CREATE TABLE Profiles (
    profile_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT,
    resume TEXT,
    skills TEXT,
    experience INT,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);

CREATE TABLE Jobs (
    job_id INT PRIMARY KEY AUTO_INCREMENT,
    employer_id INT,
    job_title VARCHAR(100),
    job_description TEXT,
    location VARCHAR(100),
    salary DECIMAL(10, 2),
    posted_date DATE,
    FOREIGN KEY (employer_id) REFERENCES Users(user_id)
);

CREATE TABLE Applications (
    application_id INT PRIMARY KEY AUTO_INCREMENT,
    job_id INT,
    applicant_id INT,
    application_date DATE,
    status ENUM('pending', 'accepted', 'rejected'),
    FOREIGN KEY (job_id) REFERENCES Jobs(job_id),
    FOREIGN KEY (applicant_id) REFERENCES Users(user_id)
);

CREATE TABLE Admins (
    admin_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    email VARCHAR(100) UNIQUE,
    password VARCHAR(100)
);

```

4.Implement Functionalities

User Registration and Login:

```
-- Registration
INSERT INTO Users (name, email, password, role) VALUES (?, ?, ?, ?);

-- Login
SELECT user_id, name, role FROM Users WHERE email = ? AND password = ?;
```

Profile Management:

```
-- Create/Update Profile
INSERT INTO Profiles (user_id, resume, skills, experience) VALUES (?, ?, ?, ?)
ON DUPLICATE KEY UPDATE resume = ?, skills = ?, experience = ?;

-- View Profile
SELECT * FROM Profiles WHERE user_id = ?;
```

Job Posting:

```
-- Post a Job
INSERT INTO Jobs (employer_id, job_title, job_description, location, salary,
posted_date) VALUES (?, ?, ?, ?, ?, CURDATE());

-- View Posted Jobs
SELECT * FROM Jobs WHERE employer_id = ?;
```

Job Search:

```
-- Search Jobs
SELECT * FROM Jobs WHERE job_title LIKE ? OR skills LIKE ? OR location LIKE ?;
```

Job Application:

```
-- Apply for a Job
INSERT INTO Applications (job_id, applicant_id, application_date, status)
VALUES (?, ?, CURDATE(), 'pending');

-- View Applications
SELECT * FROM Applications WHERE applicant_id = ?;
```

Admin Management

```
-- Admin Login
SELECT admin_id, name FROM Admins WHERE email = ? AND password = ?;

-- View All Users
SELECT * FROM Users;

-- View All Jobs
SELECT * FROM Jobs;

-- Manage Applications
UPDATE Applications SET status = ? WHERE application_id = ?;
```

5. Create User Interface

Depending on the project requirements, you might need to create a basic user interface. This could be a web-based interface using HTML, CSS, and JavaScript, or a console-based interface if you're keeping it simple.

6. Testing and Debugging

- Test each functionality thoroughly.
- Debug any issues that arise.
- Ensure the database operations are optimized and secure.

7. Documentation

- Document the database schema, API endpoints (if applicable), and usage instructions.
- Provide a README file with setup and run instructions.

SOURCE CODE

```
CREATE TABLE employers
( id INT PRIMARY KEY,
  name VARCHAR(255),
  email VARCHAR(255),
  password VARCHAR(255)
);
```

```
CREATE TABLE jobseekers
( id INT PRIMARY KEY,
  name VARCHAR(255),
  email VARCHAR(255),
  password VARCHAR(255),
  resume VARCHAR(255)
);
```

```
CREATE TABLE jobs
( id INT PRIMARY
  KEY, title
  VARCHAR(255),
  description TEXT,
  employer_id INT,
  FOREIGN KEY (employer_id) REFERENCES employers(id)
);
```

```
CREATE TABLE applications
( id INT PRIMARY KEY,
  job_id INT,
  jobseeker_id INT,
  status VARCHAR(255),
  FOREIGN KEY (job_id) REFERENCES jobs(id),
  FOREIGN KEY (jobseeker_id) REFERENCES jobseekers(id)
);
```

PHP Code for Admin

Here is the PHP code for the admin panel:

```
<?php
// admin.php

// Connect to database
$conn = mysqli_connect("localhost", "root", "", "jobportal");

// Check connection
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}

// Secure login using session
session_start();

if (!isset($_SESSION['admin'])) {
    header("Location: login.php");
    exit;
}

// Viewing all the records at once
$result = mysqli_query($conn, "SELECT * FROM jobs");
while ($row = mysqli_fetch_assoc($result)) {
    echo $row['title'] . "<br>";
}

// Filtering records based on date, position etc
if (isset($_POST['filter'])) {
    $date = $_POST['date'];
    $position = $_POST['position'];
    $result = mysqli_query($conn, "SELECT * FROM jobs WHERE date = '$date' AND position = '$position'");
    while ($row = mysqli_fetch_assoc($result)) {
        echo $row['title'] . "<br>";
    }
}

// Searching any particular form of data
if (isset($_POST['search'])) {
    $search = $_POST['search'];
```

```

$result = mysqli_query($conn, "SELECT * FROM jobs WHERE title LIKE '%$search%' OR description
LIKE '%$search%'");
while ($row = mysqli_fetch_assoc($result)) {
    echo $row['title'] . "<br>";
}
}

// Edit and Delete exclusive option
if (isset($_POST['edit'])) {
    $id = $_POST['id'];
    $result = mysqli_query($conn, "SELECT * FROM jobs WHERE id = '$id'");
    $row = mysqli_fetch_assoc($result);
    echo "<form action='edit.php' method='post'>";
    echo "<input type='hidden' name='id' value='" . $row['id'] . "'>";
    echo "<input type='text' name='title' value='" . $row['title'] . "'>";
    echo "<textarea name='description'" . $row['description'] . "</textarea>";
    echo "<input type='submit' value='Edit'>";
    echo "</form>";
}

if (isset($_POST['delete'])) {
    $id = $_POST['id'];
    mysqli_query($conn, "DELETE FROM jobs WHERE id = '$id'");
    echo "Job deleted successfully";
}

// Setting up the required spot to know the number of opening for further analysis
$result = mysqli_query($conn, "SELECT COUNT(*) as count FROM jobs");
$row = mysqli_fetch_assoc($result);
echo "Number of openings: " . $row['count'];

// Beautiful graphs, charts, bar diagrams to easily visualize the data in real-time
// (This part is not implemented in this example)

// Viewing data in individual tables
$result = mysqli_query($conn, "SELECT * FROM employers");
echo "<h2>Employers</h2>";
while ($row = mysqli_fetch_assoc($result)) {
    echo $row['name'] . "<br>";
}

$result = mysqli_query($conn, "SELECT * FROM jobseekers");
echo "<h2>Jobseekers</h2>";

```

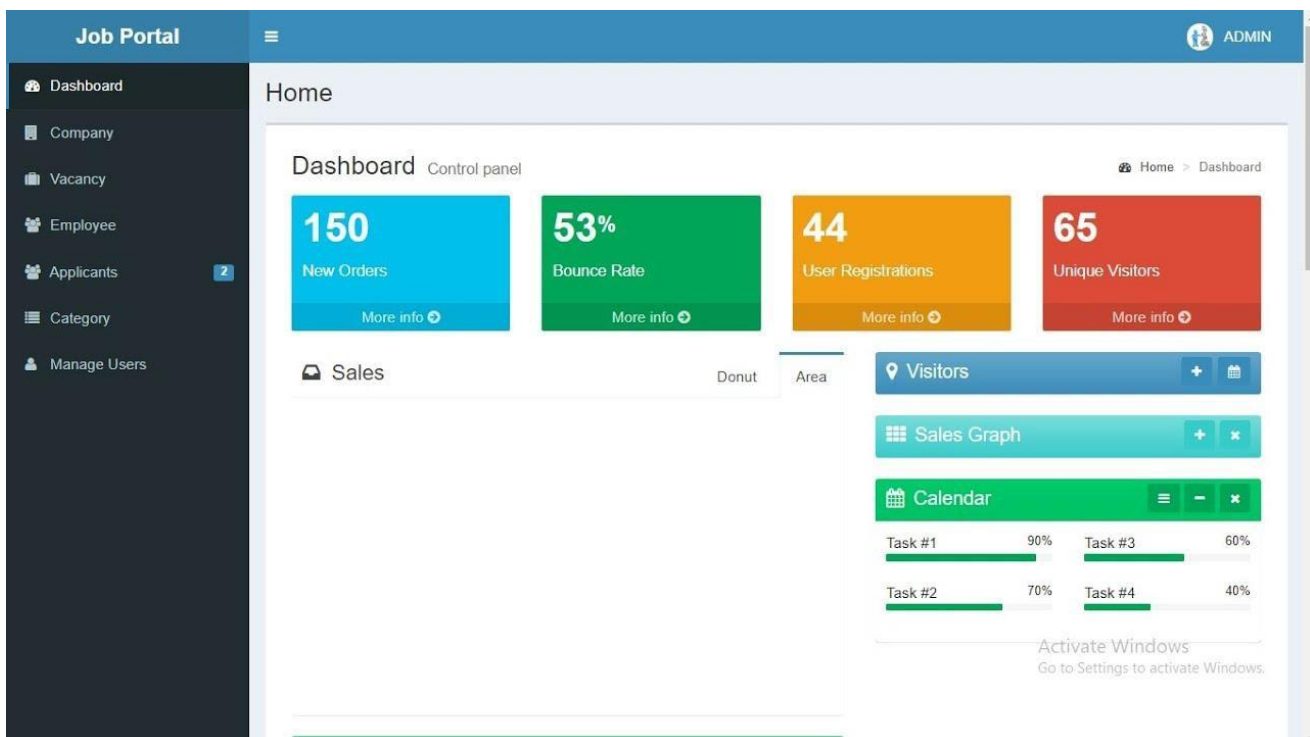
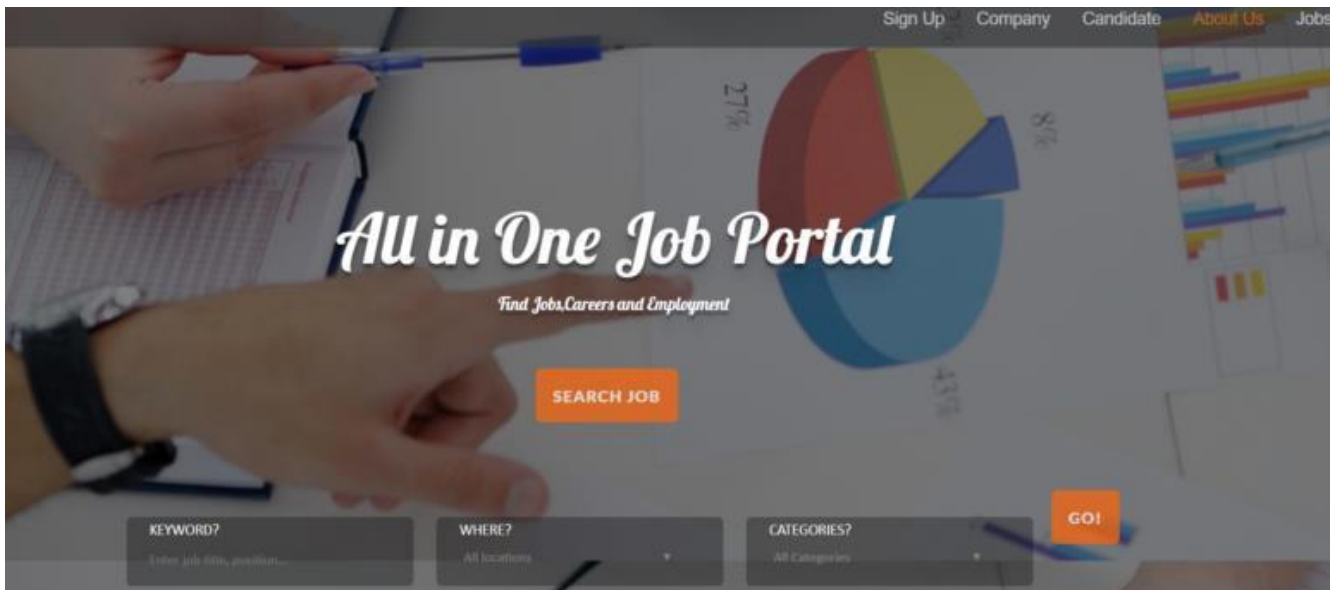
```
while ($row = mysqli_fetch_assoc($result)) {  
    echo $row['name'] . "<br>";  
}  
  
// Get today's submissions only  
$result = mysqli_query($conn, "SELECT * FROM applications WHERE date = CURDATE()");  
while ($row = mysqli_fetch_assoc($result)) {  
    echo $row['job_id'] . "<br>";  
}  
  
// Add riddles to database  
if (isset($_POST['add_riddle'])) {  
    $riddle = $_POST['riddle'];  
    mysqli_query($conn, "INSERT INTO riddles (riddle) VALUES ('$riddle')");  
    echo "Riddle added successfully";  
}  
  
?>
```

PHP Code for Jobseeker

Here is the PHP code for the jobseeker panel:

```
<?php  
// jobseek
```

SYSTEM TESTING



OUTPUT SCREEN

Applications

id	job_id	jobseeker_id	status
empty			

Customers

customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE

Employers

id	name	email	password
empty			

Jobs

id	title	description	employer_id
empty			

Jobseekers

id	name	email	password	resume
empty				

Orders

order_id	item	amount	customer_id
1	Keyboard	400	4
2	Mouse	300	4
3	Monitor	12000	3
4	Keyboard	400	1
5	Mousepad	250	2

Shippings

shipping_id	status	customer
1	Pending	2
2	Pending	4
3	Delivered	3
4	Pending	5
5	Delivered	1

SYSTEM REQUIREMENTS

Hardware Requirements:

Server:

- Processor: Intel Core i5 or higher
- RAM: 8 GB or more
- Storage: 500 GB HDD or SSD
- Network: Ethernet or Wi-Fi capability

Client:

- Processor: Intel Core i3 or higher
- RAM: 4 GB or more
- Storage: 250 GB HDD or SSD
- Network: Ethernet or Wi-Fi capability

Additional Requirements

Documentation:

- User manual
- Technical documentation

Testing:

- Unit testing
- Integration testing
- User acceptance testing

Deployment:

- Deployment scripts
- Continuous Integration/Continuous Deployment (CI/CD) setup

Software Requirements:

Operating System:

- Windows 10 or higher / macOS / Linux (Ubuntu, CentOS, etc.)

Database Management System:

- MySQL / PostgreSQL / SQLite / Microsoft SQL Server

Server Software:

- Apache HTTP Server / Nginx

Development Environment:

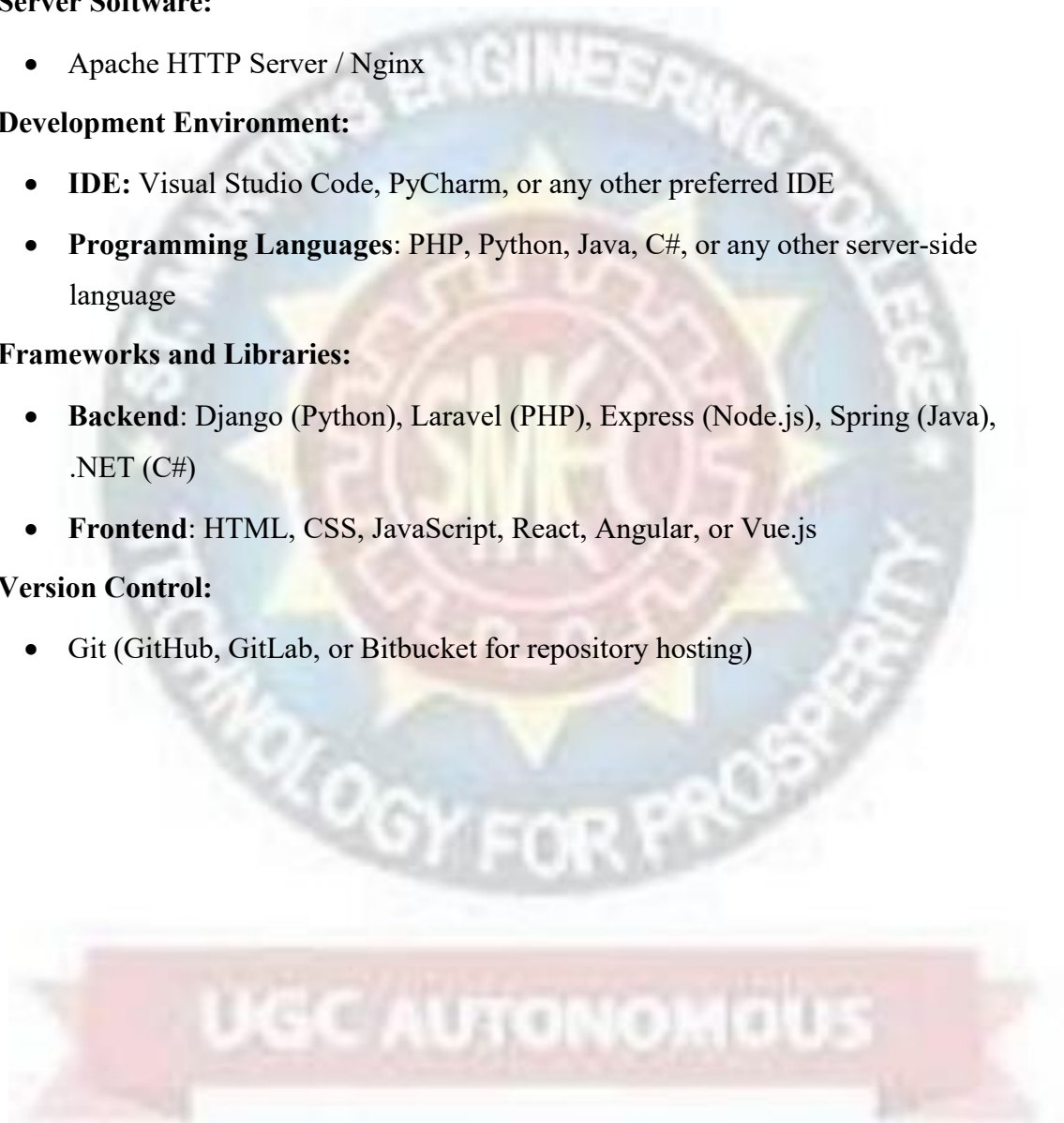
- **IDE:** Visual Studio Code, PyCharm, or any other preferred IDE
- **Programming Languages:** PHP, Python, Java, C#, or any other server-side language

Frameworks and Libraries:

- **Backend:** Django (Python), Laravel (PHP), Express (Node.js), Spring (Java), .NET (C#)
- **Frontend:** HTML, CSS, JavaScript, React, Angular, or Vue.js

Version Control:

- Git (GitHub, GitLab, or Bitbucket for repository hosting)



Functional Requirements

User Management:

- User Registration and Login
- Profile Management (for job seekers and employers)

Job Listings:

- Create, Read, Update, Delete (CRUD) operations for job postings
- Job search functionality with filters (location, job type, industry, etc.)

Application Management:

- Job application submission and tracking
- Notification system for job applications and job postings

Admin Panel:

- Manage users and job listings
- Generate reports and analytics

Security:

- User authentication and authorization
- Data encryption
- Protection against SQL injection, XSS, CSRF attacks

CONCLUSION

The conclusion of the Database Management System (DBMS) mini project on a job portal database encapsulates the core achievements and learnings gained throughout the project. By designing and implementing a robust and scalable job portal database, we successfully demonstrated the practical application of various DBMS concepts, including entity-relationship modeling, normalization, and SQL querying. The project involved creating tables for users, job listings, applications, and companies, ensuring data integrity and efficient data retrieval. Through the use of constraints, indexes, and relationships, we achieved a database structure that supports the essential functionalities of a job portal, such as user registration, job posting, job searching, and application tracking.

Furthermore, the project highlighted the importance of database security, backup, and performance optimization in a real-world application. We implemented user authentication and authorization mechanisms to protect sensitive data and conducted regular backups to prevent data loss. Performance tuning techniques, such as query optimization and indexing, were employed to enhance the efficiency of database operations. Overall, this project provided invaluable hands-on experience in database management, reinforcing theoretical knowledge with practical skills and preparing us for more complex database challenges in future endeavors. The successful completion of this project demonstrates our ability to design and manage a comprehensive database system that meets the needs of a job portal, reflecting our proficiency in DBMS.

FUTURE ENHANCEMENT

Future enhancements of the Job Portal Database management system could include the integration of advanced search and matching algorithms. These algorithms would utilize machine learning techniques to better match job seekers with suitable job openings based on their skills, experiences, and preferences. By incorporating natural language processing (NLP), the system could analyze job descriptions and resumes more effectively, making the matching process more accurate. Additionally, real-time data analytics and reporting tools could be implemented to provide employers with insights into hiring trends, candidate behaviors, and market demands, allowing them to make more informed decisions.

Another enhancement could be the development of a more interactive and user-friendly interface. Features such as chatbots for instant assistance, mobile app integration for on-the-go access, and personalized dashboards for both employers and job seekers could significantly improve user experience. Security enhancements, such as multi-factor authentication and data encryption, would ensure the protection of sensitive user information. Furthermore, the inclusion of social media integration would allow users to easily share job postings and network with potential employers or employees, thereby expanding the reach and effectiveness of the job portal.

REFERENCES

1. Elmasri, R., & Navathe, S. B. (2015). Fundamentals of Database Systems. Pearson Education.
2. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts. McGraw-Hill.
3. Date, C. J. (2019). An Introduction to Database Systems. Addison-Wesley.
4. Connolly, T. M., & Begg, C. E. (2015). Database Systems: A Practical Approach to Design, Implementation, and Management. Pearson Education.
5. Hoffer, J. A., Ramesh, V., & Topi, H. (2016). Modern Database Management. Pearson Education.
6. Kroenke, D. M., & Auer, D. J. (2013). Database Concepts. Pearson Education.
7. Ramakrishnan, R., & Gehrke, J. (2003). Database Management Systems. McGraw-Hill.
8. Coronel, C., & Morris, S. (2016). Database Systems: Design, Implementation, & Management. Cengage Learning.
9. Pratt, P. J., & Adamski, J. J. (2014). Concepts of Database Management. Cengage Learning.
10. Abraham, A., & Fong, A. C. M. (2008). Database Systems for Advanced Applications. Springer